Stone Gate Platform Architecture

An AI-native investment diligence operating system

Technical reference · Version 1.0
May 2026

Contents

1. Executive summary

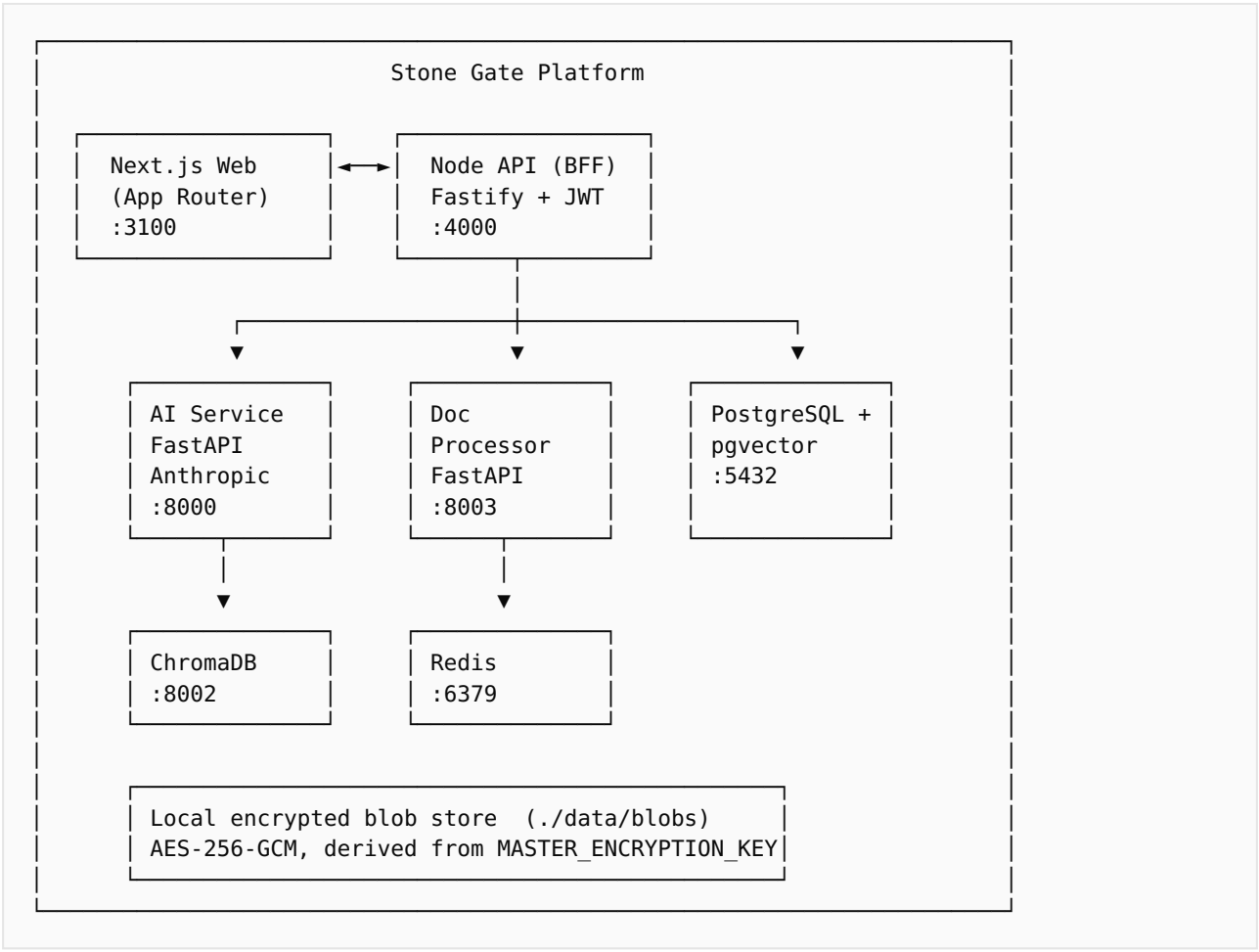
Stone Gate is a local-first, AI-native platform for institutional real estate diligence. It turns a fragmented data room (PDFs, Excel models, decks, emails) into a defensible investment recommendation through a structured workflow: **Context → Initial Screening → Understanding → CIO Review → Recommendation → Monitoring**.

The architecture is a monorepo of six runtime components:

- A Next.js web application (the analyst workspace)
- A Fastify Node API (the trust boundary and audit log)
- A FastAPI AI service (agent orchestration, RAG, financial engine, scenario engine, reporting)
- A FastAPI document processor (PDF / Excel / Word / image / email ingestion + OCR)
- PostgreSQL with pgvector (relational + embedding store)
- ChromaDB and Redis (per-opportunity vector cache and task queue)

The AI layer is a six-agent pipeline backed by Anthropic Claude with prompt caching enabled. Four agents run in parallel; two run sequentially with cross-agent context. The system produces a synthesis verdict (PROCEED / PROCEED_WITH_CONDITIONS / REJECT / NEED_MORE_INFO) plus three report formats (IC memorandum PDF, executive summary PDF, IC presentation deck).

2. System topology



Every cross-service call passes through the Node API. Service-to-service authentication uses short-lived signed tokens. The Node API is the only component the browser ever talks to directly.

3. Frontend & user interface

The web application is built on Next.js 14 (App Router) with server components for data fetching and client components for interactivity (chat streaming, charts, filters). Tailwind CSS is configured with Stone Gate design tokens; shadcn/ui primitives are extended into domain-specific components (`MetricCard`, `RiskHeatmap`, `QuadrantPanel`, `CollapsibleSection`, `ScenarioPanel`).

Routing

The opportunity workspace is organized as tabbed sub-routes following the workflow: `/opportunities/[id]/{context,initial-screening,understanding,cio-review,recommendation,monitoring}`. A computed `workflowStage` field on each opportunity drives the stepper in the header and the kanban on the pipeline page; the value is derived from the opportunity's data (does it have a thesis? a decision? approved by IC?) rather than stored as a column.

State

TanStack Query manages the server cache for opportunities, documents, threads. Per-route React state handles the rest (filters, collapsible open/closed). JWT authentication tokens are kept in `localStorage` with a 12-hour TTL. The Node API is reachable cross-origin via a `Bearer` header.

4. API gateway

The Node API in `apps/api` is the trust boundary. It enforces JWT authentication on all routes (except `/v1/auth`), records every state-changing operation to the `AuditLog` table, and forwards work to the Python services.

MODULE	RESPONSIBILITY
routes/auth	Login, logout, /me, password rotation
routes/opportunities	CRUD, briefing, analyze trigger, financial-only re-run, decision
routes/documents	Upload via multipart, processing status, soft-delete
routes/chat	Threads, message persistence, streaming proxy
routes/scenarios	Scenario CRUD, run trigger
routes/reports	Generate (IC memo / exec summary / deck), authenticated download
routes/telemetry	Aggregated LLM call counts, token usage, cost, latency
middleware/audit	Decorates every request with `req.audit(...)`

File uploads are streamed via `@fastify/multipart` with a 200 MB per-file ceiling. Uploaded blobs are AES-256-GCM encrypted at rest using a key derived from `MASTER_ENCRYPTION_KEY`; only relative paths are stored in the database so the ai-service and Node API can each resolve them against their own `BLOB_STORAGE_DIR`.

5. AI service

The FastAPI service in `services/ai-service` hosts the agent registry, orchestration logic, retrieval, financial engine, scenario engine, and report rendering. The Anthropic SDK is used directly with `max_retries=5` and a 300-second timeout to absorb transient 529 overloaded errors.

Model routing

CLASS	DEFAULT MODEL	USED FOR
`reasoning`	Claude Opus 4.7	Underwriting, thesis, risks, synthesis
`default`	Claude Sonnet 4.6	Doc classification, extraction, chat

`fast`	Claude Haiku 4.5	Categorization, summarization

Prompt caching

Anthropic's prompt caching is enabled with a *cache primer* pattern. Before the parallel agent batch fires, the orchestrator issues one tiny call that writes the document context block to the cache. The parallel agents then hit the cache instead of each writing redundant copies — saving roughly $0.19 per Analyze run with a ~0.5-second latency cost. The cached prefix is the rendered document chunks plus a shared system header; the per-agent system text sits after the breakpoint and therefore varies safely.

JSON parsing

Agent outputs are validated through a layered parser (`app/agents/_json.py`): strict `json.loads`, markdown-fence strip, smart-quote normalization, `raw_decode` longest-prefix, then a brace-balanced recovery fallback. All calls use `strict=False` to tolerate raw control characters that the model sometimes emits inside string values.

6. Document processor

The FastAPI service in `services/doc-processor` handles ingestion: detection, extraction, OCR fallback, semantic chunking, embedding, and indexing. Each blob is processed once and the chunks are stored in both pgvector and ChromaDB for hybrid retrieval (dense + BM25).

FORMAT	EXTRACTOR	NOTES
PDF	PyMuPDF + pdfplumber	Layout-aware; tables extracted separately
Excel	openpyxl + pandas	Per-sheet, per-named-range chunks
Word	python-docx	Paragraph + heading-aware
PowerPoint	python-pptx	Per-slide chunks with speaker notes
Image	Tesseract OCR	Used as fallback when PDF text extraction fails
Email	mailparser	Headers + body + attachments unpack
ZIP	zipfile (recursive)	Children dispatch back to detect

7. Data layer

PostgreSQL 16 is the system of record. The `pgvector` extension adds dense vector columns for semantic search. Prisma (in `packages/db`) is the schema and client for the Node API; the Python services use SQLAlchemy against the same database. The relational schema is the contract — both languages target it directly.

Key entities

ENTITY	PURPOSE
User · Role · AuditLog	Auth, RBAC, governance trail
Client	Investor profile, mandate, preferences
Opportunity	The deal record — thesis, scores, AI verdict, financial analysis JSON
Document · DocumentChunk	Ingested artifacts with embeddings
ExtractedMetric · FinancialAssumption	Normalized financial data points
Risk · Gap	Diligence findings (category, severity, mitigation)
Scenario · ScenarioRun	What-if analyses
ChatThread · ChatMessage · Citation	Conversational intelligence with source traceability
Report	Generated memos and decks (path + payload snapshot)
LlmCall	Per-call telemetry: model, tokens, cache hits, latency, cost

8. AI agent pipeline

The auto-Analyze pipeline that runs against an opportunity uses six agents in two stages. The first four execute concurrently via `asyncio.gather(return_exceptions=True)`; the orchestrator survives partial failures and returns whatever succeeded.

Stage 1 — Parallel (concurrent)

AGENT	MODEL	OUTPUT
Thesis Writer	Opus	Thesis paragraphs, executive summary, SWOT, bull/base/bear cases, scores (opportunity / risk / confidence 0–10)
Risk Analyst	Opus	List of risks: category, title, description, severity, likelihood, mitigation, citations
Gap Agent	Opus	Diligence gaps + IC-readiness score (0–10)
Market Researcher	Opus + web_search tool	Public-data validation of sponsor's market thesis with URL citations; TAILWIND / NEUTRAL / HEADWIND verdict

Stage 2 — Sequential

AGENT	MODEL	OUTPUT
Financial Analyst	Opus	Verdict (STRONG / MARGINAL / WEAK / INSUFFICIENT_DATA), narrative paragraphs, headline metrics, sensitivities, weak-assumption callouts, full metric extraction. Runs AFTER the parallel batch so it can interpret numbers against the risk + market findings.
Synthesis Agent	Opus	Overall AI verdict (PROCEED / PROCEED_WITH_CONDITIONS / REJECT / NEED_MORE_INFO), rationale, top reasons for / against, critical questions, next steps, watchpoints

Additional specialist agents exist as files in `app/agents/` (Document Analyst, Legal Reviewer, Scenario Agent, Portfolio Exposure, Financial Model Validator, IC Challenger, Market Stress Agent) and are dispatched by the chat orchestrator based on user intent, but they are NOT part of the auto-Analyze pipeline.

Re-run economics

A dedicated `/analyze/opportunity/financial-only` endpoint reuses the prior parallel-stage outputs from the database and re-runs only the Financial Analyst + Synthesis Agent — useful when iterating on financial prompts. Cost: ~$1 per run vs ~$3–4 for a full Analyze; latency: ~30 seconds vs ~100 seconds.

9. End-to-end flow: Analyze an opportunity

1. Analyst clicks **Analyze** on the Understanding tab.
2. Node API authenticates, audits, forwards to AI service `/analyze/opportunity`.
3. AI service retrieves the top-32 chunks via hybrid search (pgvector dense + BM25) with reciprocal-rank fusion.
4. Cache primer fires (one tiny call) to write the document context to Anthropic's prompt cache.
5. Parallel agents fire (thesis, risk, gap, market). Each reads the cache, runs ~30s.
6. Risk and Gap structured outputs are persisted to `Risk` and `Gap` tables. Thesis fields land on the `Opportunity` row.
7. Financial Analyst runs with all the above as cross-agent context.
8. Synthesis Agent runs with everything including the financial verdict.
9. Synthesis fields and `financialAnalysis` JSON are written to the Opportunity row.
10. Response returns to the browser; the page refreshes silently (no full-page loading state).

Typical total runtime: 100–130 seconds. Typical cost: $3–4 per run with cache hits.

10. Security model

- **Auth** — JWT bearer tokens, 12-hour TTL, signed with `JWT_SECRET`.
- **RBAC** — Role-based access via the `User.role` column (ADMIN, PRINCIPAL, ANALYST, ACQUISITIONS_DIRECTOR, COMPLIANCE).
- **Audit** — Every state-changing route writes an `AuditLog` entry with actor, action, entity, metadata, and IP.
- **Blob encryption** — AES-256-GCM with a per-blob nonce. Master key in `.env`; envelope unwrapping happens only inside the API services.
- **API keys** — Anthropic and Voyage keys live in `.env`, never reach the browser. The Node API proxies all Anthropic calls.
- **Data locality** — All persistent state stays on the host (Docker volumes and local disk). Only outbound LLM calls leave the machine.

11. Backup & recovery

Two PowerShell scripts under `scripts/` handle backup and restore:

- `backup.ps1` — produces a dated folder with `db.dump` (pg_dump custom-format) and `blobs.zip` (the `data/blobs` tree). Default retention: 14 backups.
- `restore.ps1` — accepts a backup folder, drops the database, restores the dump, and unzips blobs (existing blobs are moved aside, not deleted).

ChromaDB is intentionally NOT in the backup — its contents are derivable by re-ingesting documents. The `.env` is also excluded; it should live in a password manager.

12. Deployment model

Stone Gate is designed local-first. The full stack runs on a developer laptop or a single on-prem server via `docker compose up`. There is no SaaS dependency beyond the outbound Anthropic API call. The encrypted blob store, Postgres, and Redis all live on the host filesystem.

For multi-user firm deployment, the same compose file deploys cleanly behind a reverse proxy on a single VM. Horizontal scaling is not currently a requirement — the workload is single-firm, low-concurrency. The architecture leaves room for cloud single-tenant deployment in the future (Phase 4 roadmap).